

Implementing Deep Learning from First Principles: Automatic Differentiation, Optimisation and Transformers

Leuka Ye

<https://github.com/leuka-art/Building-up-to-a-transformer>

Abstract

This project explores the mathematics underpinning deep learning by implementing key components from first principles in Python using NumPy. An automatic differentiation engine, multi-layer perceptron model, optimisation algorithms and the transformer architecture are among the implemented components. The automatic differentiation engine was verified using the finite-differences method, with all tested operations achieving relative errors below 10^{-7} . The automatic differentiation engine was used to study different gradient based optimisation algorithms and investigate the significance of different transformer components through a controlled ablation study. The experiments demonstrate the impact of optimisation strategies on convergence and highlight the importance of architectural components such as residual connections and layer normalisation for stable transformer training.

1 Introduction

Modern deep learning frameworks have made the development of complex models increasingly accessible by providing high-level abstractions for automatic differentiation, optimisation, and neural network architectures. However, these abstractions can obscure the mathematical principles underlying deep learning, including gradient computation, parameter optimisation, and the role of individual model components.

This project investigates these foundations by implementing a small deep learning framework and a transformer-based language model from first principles in Python. The implementation aims to develop a deeper understanding of the computational and mathematical mechanisms that enable modern deep learning systems, while evaluating the effect of key design choices through controlled experiments.

The main objectives are:

1. Implement an automatic differentiation engine based on computational graphs to enable gradient computation through neural network operations.
2. Develop fundamental neural network components and gradient-based optimisation algorithms required for training deep learning models.
3. Construct a character-level transformer language model and investigate the contribution of key architectural components through controlled ablation experiments.

2 Background

2.1 Automatic Differentiation

Training neural networks is done by gradient descent, hence gradients need to be calculated. An efficient method to calculate gradients is automatic differentiation.

When calculating the loss for a particular batch, we can form a directed acyclic graph of the computations done. We then use the chain rule to work out gradients with respect to the required parameters.

Using matrix calculus, suppose we know $\frac{\partial L}{\partial v}$ and that $L = g(v)$, $v = x + y$ where y is a constant matrix of same shape as x .

We know $\frac{\partial v_{i,j}}{\partial x} = E_{i,j}$, where $E_{i,j}$ is a matrix with the same shape as x and is zeros in all but entry i,j .

Applying the chain rule we have $\frac{\partial L}{\partial x} = \sum_i \sum_j \frac{\partial L}{\partial v_{i,j}} \frac{\partial v_{i,j}}{\partial x} = \sum_i \sum_j \frac{\partial L}{\partial v_{i,j}} E_{i,j} = \frac{\partial L}{\partial v}$.

We can exploit the structure of the derivatives to find simple gradient update expressions. All the operation update rules were derived in a similar way to above.

A single backward pass can find all the required gradients, making it ideal for neural networks.

2.2 Neural Networks

A neural network is a parameterised function composed of multiple layers with parameters θ :

$$f_{\theta}(x) = f_n(f_{n-1}(\dots f_1(x)))$$

For input x , a linear layer is: $z = xW + b$, where W is the weight matrix and b is an optional bias vector.

A nonlinear activation function is then applied: $a = \sigma(z)$, allowing the network to model non-linear relationships. A commonly used activation function is: $ReLU(x) = \max(x, 0)$; this is due to its simplicity and that it's gradient is constant for all positive inputs (so doesn't vanish).

For classification tasks, the output probabilities are typically obtained using the softmax function:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

For classification tasks, cross-entropy loss measures the difference between predicted probabilities and target labels:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

where y is the one-hot target vector, whose i th entry indicates whether the target is class i or not. $\hat{y}_i$ is the predicted probability of being class i .

Combining these allows us to form a multi-layer perceptron model, and hence a feed forward network.

2.3 Layernorm and embedding

For input X , layernorm first z-score normalises along the feature dimension then entry-wise multiplies the feature axis by a broadcasted vector γ and adds a broadcasted vector β to each feature axis. We use the feature axis entries to approximate mean and variance.

The embedding maps an ID to a learned vector representation of chosen dimension (embedding dimension).

2.4 Attention

Transformers model sequential data using self-attention, allowing each token to incorporate information from other tokens in the sequence.

Given a token embedding, $X \in R^{B \times n \times d}$, where B is the batch size, n is the sequence length and d is the embedding dimension, the input is projected into a query, key and value representation by broadcasted linear transformations:

$$Q = XW_Q \quad K = XW_K \quad V = XW_V$$

The similarity between tokens is measured using scaled dot-product attention, where transpose swaps the last 2 axis; it is scaled to prevent softmax saturation.

$$S = \frac{QK^T}{\sqrt{d_k}}$$

The attention weights are obtained using softmax:

$$A = \text{softmax}(S)$$

and the final attention output is:

$$\text{Attention}(Q, K, V) = AV$$

This passes through another projection and is outputted.

For autoregressive language modelling, future tokens must not be visible during prediction. A causal mask is therefore applied before the softmax:

$$M_{ij} = \begin{cases} 0, & j \leq i \\ -\infty, & j > i \end{cases}$$

2.5 Transformer Model

A character-level transformer language model was implemented to investigate the components of the transformer architecture.

The model begins by converting discrete tokens into continuous vector representations. Given a token index i , the embedding layer returns the i th row of E , where $E \in R^{V \times d}$ is the embedding matrix, V is the vocabulary size, and d is the embedding dimension.

This embedding does not capture the position of the token within a sequence, hence a position embedding is added. The position embedding is indexed by the position of the token within the context window.

For a single batch, the input is a sequence of token indices, hence $X = E_{token} + E_{position}$

Following this, we compute the query, key and value and the attention operation is applied with causal mask M :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T + M}{\sqrt{d_k}}\right) V$$

This first has a linear projection applied, then the feed-forward network (FFN) is applied to the previous output. The FFN used is an MLP with output dimension d_e and a chosen hidden dimension; a linear layer followed by ReLU and then a linear layer.

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

Residual connections and layer normalisation are applied:

$$X' = \text{LayerNorm}(X + \text{Attention}(X)W) \quad Y = \text{LayerNorm}(X' + \text{FFN}(X'))$$

A transformer block consists of the attention mechanism up to and including the last layer norm. These can be stacked sequentially for a chosen number transformer blocks.

The final output is passed through a linear layer to produce logits:

$$z = W_o Y + b_o$$

These can be converted to probabilities by applying softmax.

2.6 Optimisation Algorithms

Optimisation algorithms determine how model parameters are updated during training. Given a loss function $L(\theta)$, the objective is to find parameters, θ , that minimise the loss.

For the optimisation methods, assume $g_t = \frac{\partial L}{\partial \theta}$ and η is the learning rate, where the gradient is based on the batch picked for that step (this can vary in size, which is chosen). θ_t are the parameters at step t .

The simplest optimisation method implemented was stochastic gradient descent (SGD). At each step, parameters are updated using:

$$\theta_{t+1} = \theta_t - \eta g_t$$

Although SGD is effective, it can converge slowly when gradients vary significantly between parameters. Momentum improves convergence by maintaining an exponentially weighted moving average of previous gradients:

$$v_t = \beta v_{t-1} + (1 - \beta)g_t \quad \theta_{t+1} = \theta_t - \eta v_t$$

Adaptive optimisation methods modify the learning rate independently for each parameter. Adagrad accumulates squared gradients and scales the gradient:

$$G_t = G_{t-1} + g_t^2 \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t$$

RMSProp replaces the cumulative sum with an exponential moving average:

$$v_t = \beta v_{t-1} + (1 - \beta)g_t^2 \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t + \epsilon}} g_t$$

Adadelata is an adaptive optimiser with no learning rate parameter:

$$G_t = \rho G_{t-1} + (1 - \rho)g_t^2, \quad \Delta\theta_t = -\frac{\sqrt{U_{t-1} + \epsilon}}{\sqrt{G_t + \epsilon}} g_t, \quad U_t = \rho U_{t-1} + (1 - \rho)(\Delta\theta_t)^2, \quad \theta_{t+1} = \theta_t + \Delta\theta_t.$$

Adam combines momentum and adaptive learning rates by maintaining first and second moment estimates:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t \quad \hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2 \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

giving the update:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

3 Implementation

3.1 Automatic Differentiation Engine

A tensor-based reverse-mode automatic differentiation engine was implemented using NumPy. Each tensor stores its numerical value, accumulated gradient, references to parent tensors and a local backward function defining how gradients are propagated through that operation. During the forward pass these tensors form a directed acyclic computational graph. Backpropagation traverses this graph in reverse topological order, applying the chain rule to compute gradients with respect to every trainable parameter.

The implementation was designed to support the tensor operations required by modern neural networks rather than scalar automatic differentiation. Consequently, broadcasting, batched matrix multiplication, reduction operations, reshaping and axis manipulation are supported directly, allowing the same engine to train both multilayer perceptrons and transformer models. Operations are implemented using NumPy kernels, while gradient propagation is handled entirely by the automatic differentiation engine.

Analytical gradients produced by the automatic differentiation engine were compared against numerical gradients calculated using finite differences to verify accuracy:

$$g_{num} = \frac{f(x+h) - f(x-h)}{2h}$$

The relative error was calculated as:

$$Error = \frac{\|g_{auto} - g_{num}\|}{\|g_{auto}\| + \|g_{num}\|}$$

All implemented operations achieved errors below 10^{-7} .

Table 1: Gradient checking results for the automatic differentiation engine.

Operation	Error	Operation	Error
Add scalar	1.893×10^{-11}	Broadcast row vector	3.276×10^{-12}
Multiply scalar	2.419×10^{-11}	Broadcast column vector	5.251×10^{-12}
Subtract	2.935×10^{-11}	Broadcast higher rank	1.893×10^{-11}
Divide scalar	1.716×10^{-11}	Multiply broadcast vector	2.620×10^{-11}
Power	9.778×10^{-12}	Sum operations	$< 2 \times 10^{-11}$
Exp	1.071×10^{-11}	Mean operations	$< 2 \times 10^{-11}$
Log	2.958×10^{-8}	Reshape	1.647×10^{-11}
ReLU	2.548×10^{-11}	Swap last axes	1.647×10^{-11}
Broadcast scalar add	2.712×10^{-11}	Matrix multiplication	1.212×10^{-11}
Broadcast vector	9.474×10^{-12}	Vector matrix multiplication	1.661×10^{-12}

The largest observed error occurred for the logarithm operation, with relative error 2.958×10^{-8} . This remains within an acceptable tolerance for finite-difference gradient verification.

3.2 Components

3.2.1 Module Class

The module class is a base class for all trainable modules. It has a method to return a list of references to each of the parameters in the model recursively. This allows the optimiser class later to operate on models, regardless of the architecture. The linear layer, embedding, layer norm, sequential class, multi-head attention, transformer and transformer block inherit from the module class and have their own forward passes.

3.2.2 Neural Network Components: Linear layer, Embedding, Layer norm and ReLU

The linear layer accepts tensors of shape $(\dots, d_{\text{in}})$, producing outputs of shape $(\dots, d_{\text{out}})$. Supporting arbitrary leading dimensions allows the same implementation to be used for arbitrary tensors with compatible last dimension. Batched matrix multiplication and bias broadcasting are therefore handled automatically by the Tensor class.

Weights are initialised using Xavier initialisation with variance $\frac{2}{d_{\text{in}} + d_{\text{out}}}$, which approximately preserves activation variance across layers and improves optimisation stability during the early stages of training.

The Embedding layer stores a lookup table of shape (vocabulary size, embedding dimension). Input token IDs of shape (batch size, sequence length) index the lookup table, producing an output tensor of shape (batch size, sequence length, embedding dimension). The lookup table is initialised using Xavier initialisation with variance $\frac{1}{d_e}$.

Layer normalisation was implemented using two learnable parameters, γ and β , which are broadcast across all leading dimensions. Broadcasting allows this implementation to support arbitrary batch sizes and sequence lengths. A small constant ϵ is added to the variance estimate to improve numerical stability and avoid excessively large updates when the estimated variance is close to zero.

ReLU class is a simple class which has no parameters. The output applies the `ReLU()` method to the input. It has a `parameters` method, which returns an empty list, so that it can be used in the later Sequential class.

3.2.3 Container Classes: Sequential, Multi-head Attention and Transformer

The Sequential class stores an ordered list of layers (linear, ReLU, embedding, layernorm etc.). It propagates sequentially through the list of modules. The module base class is designed to return parameters of modules from in a list, hence we can handle this module similar to other modules.

Multi-head attention stores four weight matrices initialised using Xavier initialisation. The matrices have shape (d_e, d_e) , where $d_e = h d_h$ is the embedding dimension, h is the number of heads and d_h is the head dimension. Given an input of shape (B, T, d_e) , the projections are computed in parallel, producing queries, keys and values of the shape (B, T, d_h) . These are reshaped to (B, h, T, d_h) to separate the attention heads. The attention operation is then performed using batched matrix multiplication, producing an output of shape (B, h, T, d_h) . The heads are merged by transposing and reshaping back to (B, T, d_e) . The causal mask is applied through broadcasting before the softmax operation. The final projection is applied by batch multiplication. This allows for multiple attention heads to be used in parallel; we can increase model complexity and avoid Python loops (which are slow).

The transformer block class stores the layers for a transformer block and forward propagates through them. The transformer stores the embedding, list of transformer blocks and the final linear layer.

3.2.4 Cross Entropy Loss

Although the remainder of the framework is implemented using the automatic differentiation engine, the cross-entropy calculation is performed primarily in NumPy before being wrapped as a Tensor with a custom backward function. Constructing the loss entirely from Tensor operations would create a large computational graph containing every softmax and logarithm operation. By fusing the computation into a single operation, memory usage is reduced and backpropagation becomes more efficient whilst producing identical gradients.

3.2.5 Optimisers

Each optimiser is implemented as a separate class, storing its hyperparameters and references to the model parameters. Adaptive optimisers additionally store state variables such as accumulated gradients. The `step` method updates each parameter, while `zero_grad` resets the stored gradients to 0, if required.

3.2.6 Training

Two training loops were implemented. A general purpose loop was designed for supervised learning tasks where independent samples can be shuffled each epoch. A separate transformer-specific loop was implemented because language modelling requires preserving the ordering of tokens. Rather than treating samples as independent observations, mini-batches are generated by randomly sampling context windows from the training corpus. Each input sequence is paired with a target sequence shifted by one token, enabling next-token prediction while maintaining the sequential structure of the data.

4 Experiments

4.1 Optimiser Comparison

The performance of different optimisation algorithms was evaluated using the Fashion-MNIST image classification dataset. This dataset is a more difficult classification task than MNIST, which will make it easier to see how the optimisers differ.

Fashion-MNIST consists of 70,000 greyscale images of clothing items from 10 different classes. Each image has 28×28 pixels and is flattened into a 784 dimensional input vector. Pixel values were normalised into the range $[0, 1]$ by dividing by 255. The dataset was split into equal training and validation sets using a fixed random seed.

A multilayer perceptron was trained using different optimisers while keeping the architecture, parameter initialisation and training procedure fixed. The model consisted of two hidden layers with dimensions 256 and 128 using ReLU activations, followed by a 10-class output layer. Models were trained for 20 epochs with a batch size of 64.

The optimisers compared were SGD, Momentum, Adagrad, RMSProp, Adadelata and Adam. Training loss was recorded throughout training to evaluate convergence behaviour, while final classification accuracy and validation loss were used to compare model performance.

4.2 Transformer Ablation Study

An ablation study was performed using the Tiny Shakespeare dataset to investigate the contribution of individual transformer components. The dataset consists of approximately one million characters from Shakespeare’s plays. Character-level tokenisation was used, where each unique character is assigned an integer token ID. The vocabulary contains 65 characters (letters, numbers, punctuation etc.). The Shakespeare dataset is a common benchmark and is fairly small; my limited hardware restricts the possible datasets, hence Shakespeare is used.

The tokenised text was split into training and validation sets. During training, input sequences were created by randomly sampling 64 token context windows from the training set. The target sequence was the training batch, shifted by one token, creating a next-token prediction task. Given an input sequence, the model hence learns to predict the following character at each position.

The baseline model was a character-level transformer with an embedding dimension of 64, 4 attention heads, a hidden dimension of 64, and 1 transformer block. The context length was set to 64 tokens. All models were trained for 10 epochs with 100 steps per epoch using a batch size of 64 and the Adam optimiser with learning rate 0.001.

Modified models were created by removing individual components from the baseline transformer:

- Positional embeddings
- Layer norm
- Residual connections

All models were trained under identical conditions. Performance was evaluated using training loss, validation loss, and qualitative text generation. For qualitative evaluation, each model was given the prompt “Juliet!” and used to generate 200 tokens using random sampling, not a greedy approach.

5 Results

Results shown correspond to a single training run. Due to stochastic initialisation and minibatch sampling, exact numerical values may vary between runs; however, the qualitative trends observed were consistent.

5.1 Optimiser Comparison

The curves below show the training loss over 20 epochs for each optimisation algorithm.

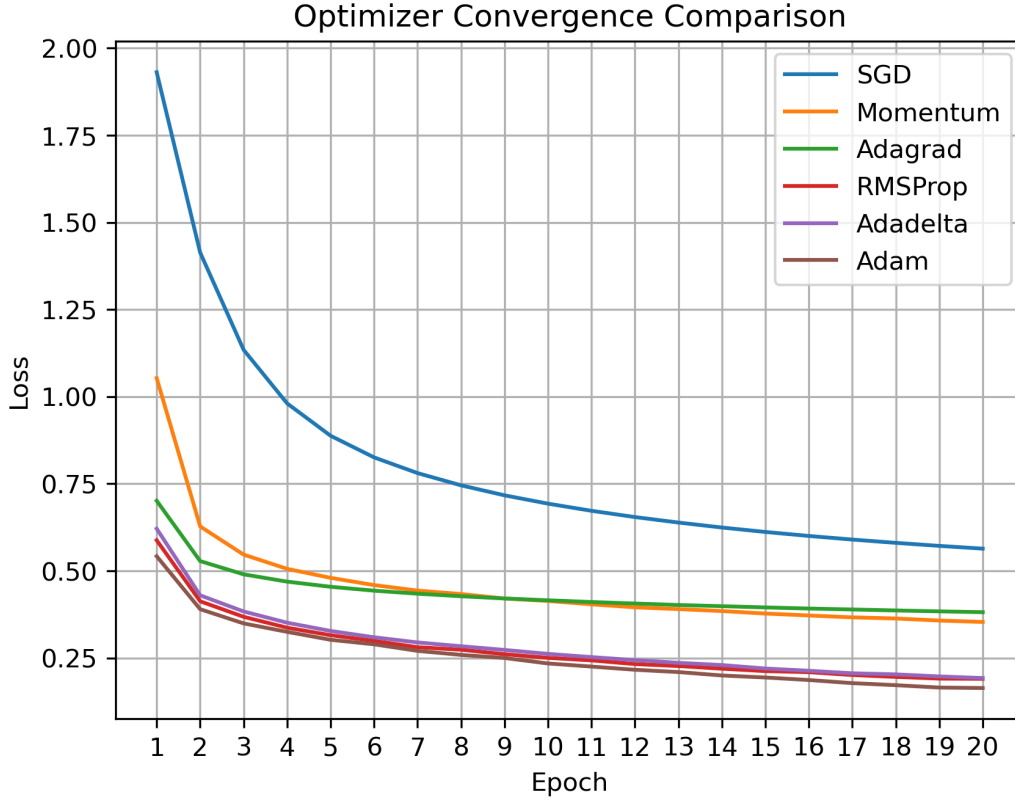


Table 2: Final performance of different optimisation algorithms on Fashion-MNIST.

Optimiser	Training Loss	Validation Loss	Validation Accuracy
SGD	0.564	0.813	81.3%
Momentum	0.354	0.387	86.1%
Adagrad	0.381	0.404	86.0%
RMSProp	0.190	0.396	88.5%
Adadelta	0.193	0.385	88.1%
Adam	0.164	0.353	89.0%

SGD is the worst performing optimiser used, with the highest losses and lowest accuracy.

Momentum improves on SGD with lower losses and higher accuracy; its loss curve shows a higher convergence rate than SGD.

Adagrad has higher losses than Momentum yet a similar validation accuracy; this demonstrates the improved performance from adaptive optimisers. We can see, however, that Adagrad’s curve has begun to flatten more than Momentum, indicating that with more epochs Momentum may be able to achieve a lower loss than Adagrad.

Adadelta and RMSProp are the next best performing optimisers with similar training and validation losses and validation accuracy; they also have similar convergence speeds to Adam, shown by the almost overlapping curves.

Adam achieved the lowest losses, as well as highest validation accuracy; it also has the highest convergence speed as seen by the curve where it reaches the lowest loss of all optimisers within a few epochs.

5.2 Transformer Ablation Study

The effect of removing individual transformer components was evaluated using the Tiny Shakespeare dataset. Figure 1 compares the training loss during optimisation for each model configuration.

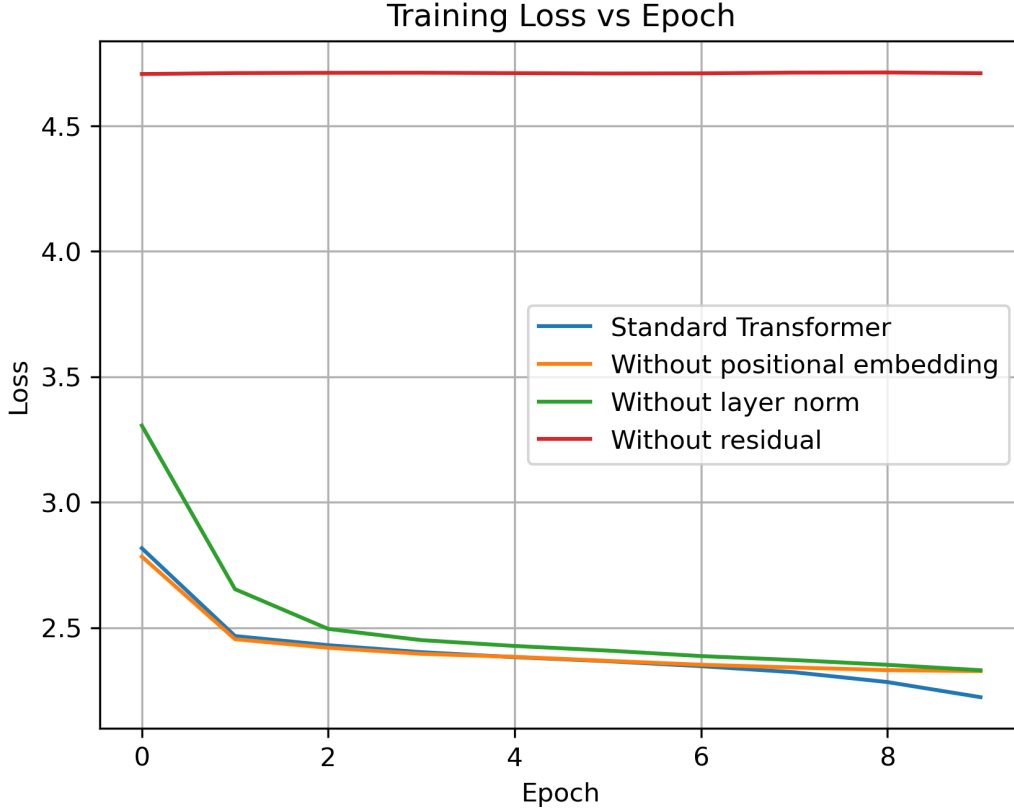


Figure 1: Training loss for the baseline transformer and the ablated models.

Table 3: Final losses for the transformer ablation study.

Model	Training Loss	Validation Loss
Standard Transformer	2.223	2.313
No positional embeddings	2.327	2.439
No layer normalisation	2.331	2.496
No residual connections	4.710	4.700

The standard transformer achieved the lowest training and validation losses, while removing residual connections resulted in the highest losses.

To qualitatively compare the models, each was prompted with “Juliet!” and used to generate the next 200 characters. The first 80 characters of generated text is shown in the next page.

The standard transformer has the lowest losses; the curve shows that the standard transformer can still further decrease in loss, showing the model easiest to optimise.

Without a positional embedding, we can see from the curve that the model begins to learn at a similar rate as a standard transformer, however, after 6 epochs the positional embedding curve begins to plateau. The final loss is hence higher than a standard transformer.

Without layer normalisation, the initial loss is much higher. We can see that the losses of this model are higher than a standard transformer.

Without residual connections, the loss is the same as randomly picking a character. It remains constant throughout training, showing that it hasn’t learned anything.

Standard Transformer

Juliet! sthak te we be Limb.
Ky My ORIN be A Muruthew pair of's mpeines
I amas...

No Positional Embeddings

Juliet! ai0 t ewnth s o, ut Y I be ta acouf tofl
'Gusutha thingoumel,
I ckind...

No Layer Normalisation

Juliet! Dervime Vomy USIVII:

A:
La y, ieeourd y herd, ber hat orgug ar tag...

No Residual Connections

Juliet! VPdgk?JK &pvAj?jIA?A&grcI-H?PmIF
IgYNQhCjYibIHYImhpxICqrTfP?hi...

6 Discussion

The experiments investigated the effect of different optimisation algorithms on MLP training and the importance of key components within the transformer architecture. The results demonstrate that the implemented framework was able to reproduce expected behaviours observed in modern deep learning systems. In particular, adaptive optimisers improved performance compared to standard SGD, whilst positional embeddings, layer normalisation and residual connections were shown to play important roles in transformer training.

6.1 Optimiser Comparison

The optimiser experiments demonstrated significant differences in optimisation behaviour between algorithms. SGD achieved the lowest validation accuracy, while adaptive methods generally converged faster and achieved improved final performance. This difference is primarily due to the way each optimiser updates parameters. Standard SGD applies the same learning rate to all parameters, which can result in inefficient optimisation when different parameters have different gradient magnitudes.

Momentum improved upon SGD by incorporating information from previous gradients. By accumulating a velocity term, momentum reduces oscillations and allows parameters to move more consistently towards a global minimum. This explains the improved convergence and validation performance compared with standard SGD.

Adaptive optimisation methods achieved further improvements by adjusting learning rates for each parameter based on the historical behaviour of gradients. Adagrad initially improved optimisation by increasing the learning rate sensitivity for infrequent parameters, however, its cumulative gradient scaling can cause learning rates to diminish over time. RMSProp and Adadelta address this limitation by using an exponential moving average of squared gradients, allowing parameters to continue making meaningful updates throughout training.

Adam achieved the best overall performance, combining adaptive learning rates with momentum-based updates. The results suggest that Adam was able to both accelerate early optimisation and maintain effective parameter updates throughout training.

6.1.1 Optimiser Comparison Limitations

The optimiser comparison has several limitations. The MLP architecture used was relatively small and may not capture optimisation difficulties present in deeper networks. Additionally, each optimiser was evaluated using a single training run, meaning variations caused by random initialisation and mini-batch ordering were not measured. A more robust evaluation would involve multiple random seeds and optimiser-specific hyperparameter tuning. Furthermore, the experiments were conducted only on Fashion-MNIST, so the observed trends may not directly generalise to other datasets or tasks. That being said, the results are consistent with modern deep learning results.

6.2 Transformer Ablation Study

The transformer ablation study investigated the contribution of positional embeddings, layer normalisation and residual connections in a character-level language model trained on the Tiny Shakespeare dataset. The baseline transformer achieved the lowest training and validation losses, demonstrating that the combination of these components provides the most effective architecture within this experimental setting.

Removing positional embeddings caused a moderate reduction in performance. Because self-attention operates on all tokens in parallel, the transformer architecture does not inherently contain information about token ordering. Without positional information, the model must infer sequence structure only from character relationships, which limits its ability to represent patterns dependent on position. The generated text remained partially coherent, suggesting that character-level dependencies could still be learned, but the increased validation loss indicates that positional information improves sequential modelling.

Removing layer normalisation also reduced performance and slowed convergence. Without it, the model was still able to learn meaningful language patterns, but the higher initial loss and slower convergence suggest that optimisation became more difficult. This demonstrates that layer normalisation primarily improves training stability rather than directly providing additional modelling capacity.

The removal of residual connections produced the greatest degradation in performance. Both training and validation losses remained substantially higher than the other models and the generated text showed little evidence of learned language structure. Residual connections provide a more direct backward path through the computation graph; this greatly reduces the chance of vanishing gradients and makes optimisation easier.

The qualitative generation results support the quantitative loss measurements. The baseline transformer generated the most coherent text, whilst progressively removing architectural components resulted in increasingly degraded outputs. The agreement between validation loss and generated samples suggests that the ablation experiments successfully captured the contribution of each component to language modelling performance.

6.2.1 Transformer Study Limitations

The transformer experiments were limited by the size of the implemented architecture. Due to hardware constraints, the model consisted of a single transformer block with a relatively small embedding dimension, limiting the complexity of the generated text. Larger transformer models would likely produce more coherent outputs and may exhibit different scaling behaviour.

Additionally, each experiment was performed using a single random initialisation and fixed hyperparameters. Repeating experiments across multiple seeds and performing hyperparameter optimisation would provide a more reliable estimate of the effect of each architectural component. However, the large performance differences observed, particularly for residual connections, suggest that the main conclusions are unlikely to change.

7 Conclusion

This project developed a small deep learning framework from first principles and used it to investigate the practical behaviour of optimisation algorithms and transformer architectures. The implementation included automatic differentiation, neural network layers, optimisation algorithms and a transformer model capable of character-level language modelling.

The optimiser experiments demonstrated that adaptive optimisation methods generally achieved faster convergence and improved performance compared with standard SGD, with Adam achieving the strongest results in this experimental setting. The transformer ablation study showed that positional embeddings, layer normalisation and residual connections each contribute to effective training, with residual connections having the largest impact on optimisation and model performance.

Overall, this project provided insight into the mathematical and computational foundations of modern deep learning architectures. Building these components from scratch highlighted how theoretical concepts such as gradient propagation, layer normalisation and attention mechanisms directly influence the behaviour of practical machine learning systems.